STATE MIND

Symbiotic Core V2

28-05-2026 - 26-06-2026

Table of contents

1. Project brief

Title	Description
Client	Symbiotic
Project name	Symbiotic Core V2
Timeline	28-05-2026 - 26-06-2026

Project Log

Date	Commit Hash	Note
08-06-2026	2ec9358d2f8bb81f9d54ead176b8d4c043f260af	Initial Commit
15-06-2026	1b6fd6385763fe44d96d46aaecd9364198030149	Initial commit for the scope extension
22-06-2026	5cba0844ba143db893692bb9079e642a042ed5d7	Reaudit commit for initial scope
24-06-2026	21827eade2847098ebb55962d1b2c0f59aa226b7	Reaudit commit for the scope extension
25-06-2026	07870e8aafb42d3863cfd689d69f780ba38866a1	Reaudit commit 2 with migratable delegator
26-06-2026	f32eec02c88184af2641e07100ba1a57c734e53e	Reaudit commit 2 for the scope extension
26-06-2026	b43a1f63fd30def67a1fb1a7a8bd14cf3d7a8ae9	Reaudit commit 3 for the scope extension

Short Overview

Symbiotic Core V2 introduces a redesigned Symbiotic core architecture focused on modular asset allocation, adapter-based integrations, and simplified withdrawal management. The audited scope includes the Vault V2 system, Universal Delegator, and adapters.

New features include:

- Vault V2
  - ERC4626 vault responsible for deposits, withdrawals, share and asset accounting, and fee accrual.
  - Supports both instant withdrawals and queued withdrawals when instant liquidity is insufficient. Queued withdrawal logic is processed through a separate WithdrawalQueue contract.
  - Removes the fixed withdrawal delay used in Vault V1; withdrawal timing now depends on liquidity availability and connected adapters.
- Universal Delegator
  - Manages asset allocation and deallocation across connected adapters.

    - Enforces per-adapter allocation limits and controls allocation/deallocation ordering.
    - Maintains total asset accounting while keeping allocations to different adapters isolated from each other.

- Adapters
    - Represent the asset deployment layer of the Vault V2 system.
    - Encapsulate integration-specific logic for external DeFi protocols or applications requiring slashable stake-time guarantees.
    - Must implement the predefined interface and behavior expected by the Universal Delegator.
    - App Adapter is a specialized adapter for applications that require slashable stake-time guarantees. The application can consume these guarantees for a configured duration, while the adapter preserves the required stake-time conditions and provides slashing functionality when applicable.

Project Scope

The audit covered the following files:

CoWSwapConverter.sol	MerklClaimer.sol	RestakingAppAdapter.sol
VaultV2.sol	WithdrawalQueue.sol	WithdrawalQueueFactory.sol
UniversalDelegator.sol	AdapterRegistry.sol	ProtocolFeeRegistry.sol
AdapterFactory.sol	Adapter.sol	AppAdapter.sol
MorphoVaultV2Adapter.sol	AaveV3Adapter.sol	

2. Finding severity breakdown

All vulnerabilities discovered during the audit are classified based on their potential severity and have the following classification:

Severity	Description
Critical	Bugs leading to assets theft, fund access locking, or any other loss of funds to be transferred to any party.
High	Bugs that can trigger a contract failure. Further recovery is possible only by manual modification of the contract state or replacement.
Medium	Bugs that can break the intended contract logic or expose it to DoS attacks, but do not cause direct loss of funds.
Informational	Bugs that do not have a significant immediate impact and could be easily fixed.

Based on the feedback received from the Client regarding the list of findings discovered by the Contractor, they are assigned the following statuses:

Status	Description
Fixed	Recommended fixes have been made to the project code and no longer affect its security.
Acknowledged	The Client is aware of the finding. Recommendations for the finding are planned to be resolved in the future.

3. Summary of findings

Severity	# of Findings
Critical	0 (0 fixed, 0 acknowledged)
High	0 (0 fixed, 0 acknowledged)
Medium	5 (5 fixed, 0 acknowledged)
Informational	36 (14 fixed, 22 acknowledged)
Total	41 (19 fixed, 22 acknowledged)

4. Conclusion

During the audit of the codebase, 41 issues were found in total:

- 5 medium severity issues (5 fixed)
- 36 informational severity issues (14 fixed, 22 acknowledged)

The final reviewed commit is b43a1f63fd30def67a1fb1a7a8bd14cf3d7a8ae9

5. Findings report

MEDIUM-01	Unclearable deallocation debt strands slashable assets in **AppAdapter**	Fixed at: 6dd8d48

Description

Line: AppAdapter.sol#L170-L188

AppAdapter represents the network guarantee with a **Stake { initialStake, slashed, debt }**, where the live guarantee is:

```
slashable() = initialStake - slashed - debt   // debt = matured cumulative debt
```

debt is a cumulative checkpoint trace: **debt.latest()** is the total scheduled deallocation, and slashable is reduced by it. To wind the guarantee down, **UniversalDelegator** calls **_requestDeallocate(adapter, amount)**, where **amount** is the size of the current deallocation request (how much this call wants moved out), recomputed from scratch on each call as **min(remainingPendingAssets, totalAssets())** (UniversalDelegator.sol#L404). It is therefore an incremental, non-cumulative quantity.

```
function _requestDeallocate(uint256 amount) internal virtual override {
    uint256 curSlashable = _slashable();

    if (
        Math.min(limitOf(address(this)), totalAssets()).saturatingSub(amount) >= curSlashable
    ) {
        // RESET branch: rebase into a fresh stake (debt = 0). Gated by limitOf.
        _stakePos.push(uint48(block.timestamp), uint208(_stakes.length));
        _stakes.push().initialStake = curSlashable;
    } else {
        // ELSE branch: bump debt. Gated by debt.latest(), which is cumulative.
        Stake storage curStake = _stakes[_stakePos.latest()];
        if (curStake.debt.latest() < amount) {
            curStake.debt.push(uint48(block.timestamp) + duration, amount);
        }
    }
}
```

There are two gates, and both can be closed simultaneously:

1. **limitOf** gates the storage reset. The RESET branch is the clean way to lower slashable: it starts a new stake whose **initialStake = curSlashable** with **debt = 0**. Because **sweepPending()** always sweeps free assets before calling **_requestDeallocate()**, the adapter's remaining assets are entirely slashable, so **totalAssets() == curSlashable** at the call site. The guard **min(limitOf, totalAssets) − amount >= curSlashable** then collapses to **min(limitOf, curSlashable) − amount >= curSlashable**, which cannot hold once **limitOf < curSlashable** (even for **amount == 0**). A curator lowering the absolute limit, or a sub-100% share cap, closes this gate.
2. **debt.latest()** gates the debt bump, and it is cumulative — but **amount** is not. In the ELSE branch, the only write is

```
if (curStake.debt.latest() < amount) {
    curStake.debt.push(uint48(block.timestamp) + duration, amount);
}
```

The new debt is stored as the raw **amount** and the bump is allowed only when **amount** exceeds the current cumulative **debt.latest()**. This is the core defect: **amount** is the size of a single deallocation request, while **debt.latest()** is the running cumulative total that slashable is reduced by. Once any debt has accumulated, the cumulative **debt.latest()** exceeds a later,

smaller per-request **amount**, the guard **debt.latest() < amount** is false, and the bump is skipped — so the slashable amount cannot be reduced even though those assets should be deallocatable.
With gate 1 closed by **limitOf** and gate 2 closed because the incremental **amount** is smaller than the cumulative **debt.latest()**, neither branch can reduce slashable, and the guarantee becomes stuck.

Step-by-step example

1. Initial: **initialStake = 100**, **slashed = 0**, **debt = 0** → **slashable = 100**, **totalAssets = 100**.
2. Curator winds down and sets the absolute limit to 30 → **limitOf = 30 < slashable = 100**.
3. User redeems 60 → **_requestDeallocate(adapter, 60)**:
    - RESET guard: **min(30, 100) − 60 = 0 >= 100** → false → ELSE.
    - ELSE: **debt.latest() 0 < 60** → push **debt = 60**, maturing at **now + duration**.
4. After **duration** the debt matures: **slashable = 100 − 0 − 60 = 40**, **freeAssets = 100 − 40 = 60**.
5. **sweepPending()** sweeps the 60 free assets out → **totalAssets = 40**. State: **initialStake = 100**, **slashed = 0**, **debt = 60**, **slashable = 40**, **limitOf = 30**.
6. The residual 40 is still slashable and must be deallocated to service further withdrawals → **_requestDeallocate(adapter, 40)** (or the **amount == 0** clear at L423):
    - RESET guard: **min(30, 40) − 40 = 0 >= 40** → false (gate 1 closed by **limitOf**).
    - ELSE: **debt.latest() 60 < amount 40** → false → nothing happens (gate 2 closed: incremental 40 < cumulative 60).
    - To fully deallocate the residual, the cumulative debt would need to reach 100 (driving slashable to 0), but the request only carries the incremental **amount = 40**, so the bump is rejected.

The adapter is stuck: **slashable = 40** cannot be reduced. **deallocate(adapter, max)** returns **0**, and reallocation cannot heal it because **limitOf(30) < totalAssets(40)** makes **allocatable() == 0**.
The curator can escape from the deadlock state by increasing a limit, which allows calling the adapter allocation to reset the state. The network's **release()** can also free the residual.
Impact: Residual slashable assets become undeallocatable. Any withdrawal that needs those assets is stuck.
As shown in **test_AttackerDoSsWithdrawals()**, under a normal sub-100% share cap, this state is reachable permissionlessly by any user via a partial redeem.

Recommendation

We recommend converting the request into the same cumulative units as **debt** before storing or comparing it

MEDIUM-02	Proposed **AppAdapter._requestDeallocate()** fix drops **limitOf** gate and lets **UniversalDelegator.sweepPending()** wipe immature debt	Fixed at: 2916b37

Description

Lines:

- AppAdapter.sol#L170-L188
- UniversalDelegator.sol#L314-L340
- UniversalDelegator.sol#L414-L425

A previously reported issue identified that the **limitOf** gate in **AppAdapter._requestDeallocate()** blocks stake reset after **forceDeallocate**. A fix was proposed:

```
function _requestDeallocate(uint256 amount) internal virtual override {
        uint256 curSlashable = _slashable();
        uint256 targetSlashable = totalAssets().saturatingSub(amount);


    // Reset stake, debt, and slashed when the deallocation fits within the free (non-slashable) assets.
    if (targetSlashable >= curSlashable) {
        uint256 limit = IUniversalDelegator(IVaultV2(vault).delegator()).limitOf(address(this));
        if (limit >= curSlashable && limit < targetSlashable) {
            targetSlashable = limit;
        }

        _stakePos.push(uint48(block.timestamp), uint208(_stakes.length));
        _stakes.push().initialStake = targetSlashable;
    } else {
        Stake storage curStake = _stakes[_stakePos.latest()];

        // Keep post-maturity slashable equal to the assets left after this request:
        // debt = (initialStake - slashed) - (totalAssets - amount).
        uint256 targetDebt =
            curStake.initialStake.saturatingSub(curStake.slashed.latest()).saturatingSub(targetSlashable);

        // Keep increasing debt when the request grows.
        if (curStake.debt.latest() < targetDebt) {
            curStake.debt.push(uint48(block.timestamp) + duration, targetDebt);
        }
            // Keep existing debt when the request shrinks but cannot release the amount yet.
        }
    }
```

It replaces the condition with **targetSlashable >= curSlashable**, removing the **limitOf** dependency. That unblocks debt accounting when **limitOf** was the sole obstacle, but it reintroduces a regression on the cleanup path:

1. **WithdrawalQueue.requestRedeem()** schedules WQ debt and puts the adapter in **adaptersWithPending**.
2. **UniversalDelegator.forceDeallocate()** pushes or retains immature debt, then calls **UniversalDelegator.sweepPending()**.
3. **UniversalDelegator.sweepPending()** fills the queue (liquidity may come from **VaultV2.freeAssets()**, another adapter via **UniversalDelegator._deallocateAll()**, or both).
4. When **WithdrawalQueue.pendingAssets() == 0**, the adapter drops out of **adaptersWithPending** and cleanup invokes **AppAdapter._requestDeallocate(0)**.
5. With the proposed condition, **targetSlashable = totalAssets()** and **curSlashable** are equal while debt is still immature (**AppAdapter._slashable()** does not yet count the checkpoint). The reset branch runs, pushes a fresh stake, and wipes the curator's debt.

Step-by-step example

Initial (t=100): **initialStake=100**, **slashed=0**, **debt=0** -> **stake()=100**, **totalAssets=100**, **vault.freeAssets=0**.

1. Alice **WithdrawalQueue.requestRedeem(20 shares)** - **sweepPending** triggered internally.
    - Vault has no free assets, fill() fails;
    - **AppAdapter._requestDeallocate(20)**: **curSlashable=100**, **targetSlashable=50 < 100** -> ELSE branch;
    - **targetDebt=100–0–20=80**; **debt.latest()=0 < 20** -> push **debt=20** at key **t+duration=110**. Adapter enters **adaptersWithPending**.
2. t=101: **stake()** uses **debt.upperLookupRecent(101+10–1=110)=20** -> **stake()=80**. **_slashable()** uses **debt.upperLookupRecent(101)=0** (key 110 not yet reached) -> **curSlashable=100**.
3. Curator **UniversalDelegator.forceDeallocate(adapter, 80)** — **_requestDeallocate(80)**:
    - **curSlashable=100**, **targetSlashable=20 < 100** -> ELSE;
    - **targetDebt=80; debt.latest()=20 < 80** -> push **debt=80** at key **t+duration=111**. **_setLimits(adapter, 20, ...)**;
    - **UniversalDelegator.sweepPending()** inside: vault still empty -> **WithdrawalQueue.fill()** fails -> **debt=80** survives. **stake()** at **t=102** -> **debt.upperLookupRecent(111)=80** -> **stake()=20**.
4. Bob deposits 100 assets — vault gains 100 free tokens. **UniversalDelegator.onDeposit()** -> **UniversalDelegator.sweepPending()** -> **WithdrawalQueue.fill()**:
    - **withdrawable=100**, **previewDeposit(100)≥pendingShares=50** -> WQ fully filled;
    - Adapter removed from **adaptersWithPending** -> cleanup: **AppAdapter._requestDeallocate(0)**;
    - **_slashable()** at t=102: **debt.upperLookupRecent(102)=0** (keys 110, 111 not yet reached) -> **curSlashable=100**;
    - **targetSlashable=totalAssets()=100**; **100 ≥ 100** -> RESET - new stake checkpoint **initialStake=100**, **debt=0**;
    - Curator's **debt=80** wiped;
    - **stake()=100**.
5. t=112: **stake()=100**, **freeAssets=0**.

Expected: after forceDeallocate(80), network guarantee drops to 20; 80 tokens freed after maturity. Actual: Bob's routine deposit wipes debt=80 -> **stake()** never reduces -> network sees full 100 as guaranteed.

Recommendation

We recommend not adopting a reset condition based only on **targetSlashable >= curSlashable**. If **limitOf** is removed from the gate, either skip the reset branch in **AppAdapter._requestDeallocate(0)** while immature curator's debt exists, or split WQ debt from curator **forceDeallocate** debt so WQ cleanup cannot clear unrelated reservations.

MEDIUM-03	Matured AppAdapter withdrawal debt is not settled before slash	Fixed at: e63ec83

Description

Lines:

- AppAdapter.sol#L103
- WithdrawalQueue.sol#L133

When a user calls **WithdrawalQueue.requestRedeem()**, **AppAdapter._requestDeallocate()** schedules a debt checkpoint at **block.timestamp + duration**. Once that time elapses, the debt is included in **AppAdapter._slashable()** (via **debt.upperLookupRecent(block.timestamp)**), so those assets are no longer slashable — the user's exit window has expired. However, **AppAdapter.slash()** does not call **UniversalDelegator.sweepPending()** before executing. The user's pending shares remain in the **WithdrawalQueue** and have not yet been redeemed via **WithdrawalQueue.fill()**. **WithdrawalQueue.fill()** redeems at the current vault share price (**previewRedeem(pendingShares())**), so a slash that executes between debt maturation and **WithdrawalQueue.fill()** deflates the share price — and the user's shares, despite having matured beyond the slash window, are converted to fewer assets than expected.

Impact: A user whose withdrawal request has fully matured (i.e., **duration** has elapsed since **WithdrawalQueue.requestRedeem**) may receive fewer assets than their shares were worth at request time, because a slash between debt maturation and queue settlement reduces the vault exchange rate before their shares are redeemed.

Recommendation

We recommend calling **sweepPending()** at the start of **AppAdapter.slash()**, so that matured pending shares are filled at the pre-slash exchange rate before any slash accounting occurs:

```
    function slash(uint256 amount) public virtual returns (uint256) {
        if (INetworkMiddlewareService(NETWORK_MIDDLEWARE_SERVICE).middleware(subnetwork.network()) !=
msg.sender) {
            revert NotNetworkMiddleware();
        }
+       IUniversalDelegator(IVaultV2(vault).delegator()).sweepPending();

        amount = Math.min(amount, _slashable());
```

MEDIUM-04	invalidateConvert() can clear a signed order but not a pending prepared reservation	Fixed at: a8ab604

Description

Lines:

- CoWSwapConverter.sol#L166-L173
- CoWSwapConverter.sol#L136-L154

CoWSwapConverter.invalidateConvert() lets a converter clear the CoW pre-signature of an already-signed order UID. There is no corresponding way to cancel a reservation created by **CoWSwapConverter.prepareConvert()** before it becomes executable: the **executableAt** entry can only be consumed by **CoWSwapConverter.convert()** after the delay, and is otherwise never deleted. A reservation prepared with undesired parameters, or one that becomes undesirable during the **EXECUTION_DELAY**, cannot be revoked; the available options are to let it execute after the delay or to leave it occupying the slot.

Impact: The protocol cannot abort a maturing reservation, including one prepared with adverse parameters, before it becomes executable.

Recommendation

We recommend adding a cancellation entry point that deletes **executableAt[nonces(tokenIn)][requestHash]** for a pending reservation, gated to converters (and/or to the original preparer) so it cannot be used to grief honest reservations.

MEDIUM-05	Unbounded claimable backlog in **syncSlash()** lets an attacker brick slashing	Fixed at: f32eec0

Description

Lines:

- RestakingAppAdapter.sol#L130-L158
- RestakingAppAdapter.sol#L165-L174

RestakingAppAdapter.syncSlash() walks the per-vault withdrawal-request list and, for each underlying vault, greedily claims pending requests in a **for** loop that has no iteration cap. The loop only stops early when a request is not fully claimable.

The early **break** relies on the underlying **WithdrawalQueue** filling FIFO, so that an unfilled request stops the traversal and bounds the work per call. This bound disappears when every queued request becomes claimable in the same block: the loop then has to iterate the entire accumulated backlog in a single transaction.

An arbitrary user can manipulate the vault's state to build an inescapable backlog, if conditions are met:

- The adapter sits on top of a multi-level restaking chain (**T -> A -> B -> base**) with at least two underlying vaults.
- Two consecutive underlying vaults (**A** and **B**) hold no free liquidity, so requests against them cannot be filled while the attacker grows the backlog.
- A pending slash request already exists against vault **A** (created by a normal slash while **A** is illiquid).

```
// Create a request for previously claimed vault shares.
if (amount > 0) {
    withdrawalRequests[vault].tokenIds
        .push(uint64(IWithdrawalQueue(withdrawalQueue).requestRedeem(amount, address(this))));
    amount = 0; // ^-- for each dust 'B' shares donation creates a request
}
// Try claim existing requests greedily.
WithdrawalRequests storage requests = withdrawalRequests[vault];
uint256 indexToClaim = requests.firstUnclaimed;
for (; indexToClaim < requests.tokenIds.length; ++indexToClaim) {
    uint256 tokenId = requests.tokenIds[indexToClaim];
    try IWithdrawalQueue(withdrawalQueue).claim(tokenId, address(this)) returns (uint256 assets, uint256) {
        amount += assets; // ^-- deposit dust 'B' shares to create lots of small requests
    } catch {}
    // Stop if the last request was not fully claimed.
    if (!IWithdrawalQueue(withdrawalQueue).isClaimed(tokenId)) {
        break; // ^-- can create 1 request per ùyncSlash()`call. Donate at the end to stop escaping for vault B`
    }
}
requests.firstUnclaimed = indexToClaim;
```

1. Grow the backlog (permissionless, ~free). For each cycle the attacker donates **DUST** of vault **B** shares into **A**, calls **delA.sweepPending()** to fill **A**'s pending request by **DUST**, then calls **syncSlash()**. **syncSlash()** claims that **DUST** from **A** and immediately calls **requestRedeem(amount)** on **B**, pushing one new **tokenId** into **withdrawalRequests[B].tokenIds**. Because **B** stays illiquid, nothing is claimed and the list only grows. Repeating the cycle appends an arbitrary number of requests cheaply.
2. Make the backlog claimable in one shot. The attacker makes vault **B** liquid with a single base-asset deposit. The deposit's **onDeposit** hook sweeps and fills **B**'s queue, turning the entire backlog claimable at once.
3. Trigger the unbounded traversal. The next **syncSlash()** can no longer take the early **break** and must claim every request in a single transaction. Once that traversal exceeds the gas cap, the call cannot be included in any valid transaction.

slash() and **release()** both begin with a call to **syncSlash()**:

Impact: A permissionless actor can grow the claimable withdrawal backlog until **syncSlash()** no longer fits within the per-transaction gas cap. The network can be prevented from slashing or releasing the adapter, disabling the core penalty

mechanism for the affected restaking position.
PoC was provided to the client.

Recommendation

We recommend bounding the work performed per call so that **syncSlash()** cannot be pushed past the per-transaction gas limit by backlog growth.

INFORMATIONAL-01	Loop gas optimizations	Fixed at: ac71fb7

Description

Lines:

- UniversalDelegator.sol#L385-L387
- UniversalDelegator.sol#L227-L235
- UniversalDelegator.sol#L182-L188
- UniversalDelegator.sol#L97-L99
- UniversalDelegator.sol#L402-L411
- UniversalDelegator.sol#L414-L421
- UniversalDelegator.sol#L461-L465
- UniversalDelegator.sol#L470-L474

Several loops over storage arrays read the array length from storage in the loop condition on every iteration, and in some cases, read the same indexed storage element multiple times within the body. Each such access is a separate **SLOAD** that could be cached once in memory and reused.

1. In **UniversalDelegator.sweepPending()**, the loop re-reads the storage array length **adapters.length** in the condition on every iteration, and reads the storage element **adapters[i]** twice within the body.
2. In **UniversalDelegator.swapAdapters()**, the loop re-reads the storage array length **adapters.length** in the condition on every iteration, and reads the storage element **adapters[i]** twice within the body.
3. In **UniversalDelegator.removeAdapter()**, the loop re-reads the storage array length **adaptersWithPending.length** in the condition on every iteration and again in the body (**adaptersWithPending[adaptersWithPending.length – 1]**).
4. In **UniversalDelegator.totalAssets()**, the loop re-reads the storage array length **adapters.length** in the condition on every iteration.
5. In the second loop of **UniversalDelegator.sweepPending()**, the loop re-reads the storage array length **adapters.length** in the condition on every iteration.
6. In the reset loop of **UniversalDelegator.sweepPending()**, the inner loop re-reads the storage array length **adaptersWithPending.length** in the condition on every iteration.
7. In **UniversalDelegator._allocateAll()**, the loop re-reads the storage array length **autoAllocateAdapters.length** in the condition on every iteration.
8. In **UniversalDelegator._deallocateAll()**, the loop re-reads the storage array length **adapters.length** in the condition on every iteration.

Recommendation

We recommend caching the array element and length.

INFORMATIONAL-02	Unused declarations in the interfaces and contracts	Fixed at: 238e06e

Description

Lines:

- VaultV2.sol#L53
- IVaultV2.sol#L44
- IVaultV2.sol#L54
- IVaultV2.sol#L69
- IVaultV2.sol#L79
- IVaultV2.sol#L84
- IVaultV2.sol#L136
- IWithdrawalQueue.sol#L35
- IUniversalDelegator.sol#L52
- IUniversalDelegator.sol#L83
- IUniversalDelegator.sol#L93
- IUniversalDelegator.sol#L190

Across the V2 vault, withdrawal queue, and delegator surface, several declarations are unused in the corresponding implementations. These include custom **error** definitions, **event** declarations, and an unused **using** directive for a checkpoint trace type in **VaultV2** while only the other trace type is actually referenced. This creates a misleading API.

Recommendation

We recommend removing unused declarations from the interfaces and contracts.

INFORMATIONAL-03	Missing early reverts for zero-amount and no-op paths waste gas	Acknowledged

Description

Lines:

- VaultV2.sol#L299-L302
- VaultV2.sol#L305-L309
- AppAdapter.sol#L127-L148

1. **VaultV2.withdraw()** and **VaultV2.redeem()** have no zero-input guard. A call with **assets == 0** or **shares == 0** runs the full OZ **ERC4626** path, including the **sweepPending()** guard in **_withdraw()**.
2. **AppAdapter.release()** does not guard against the **amount** being capped to **0** by **Math.min(amount, _slashable())**. It still writes a duplicate checkpoint to **slashed**, calls **UniversalDelegator.decreaseLimits()**, and emits **Release(0)**.

Recommendation

We recommend the following changes:

1. Add an explicit revert (or early return) at the top of **VaultV2.withdraw()** and **VaultV2.redeem()** when **assets == 0** or **shares == 0**.
2. Revert or return early in **AppAdapter.release()** when **amount == 0** after **Math.min(amount, _slashable())**.

Client's comments

> Prefer minimalistic code style, while such cases do not harm common operations.

INFORMATIONAL-04	WithdrawalQueue.isClaimed() returns true for non-existent tokenId	Fixed at: 80329e8

Description

Lines: WithdrawalQueue.sol#L64-L67

WithdrawalQueue.isClaimed() compares **request.sharesClaimed == request.shares** without checking that the NFT exists. For a non-existent **tokenId**, both fields default to **0**, so the function returns **true**.

Recommendation

We recommend returning **false** when **tokenId >= _nextTokenId** before comparing **sharesClaimed** and **shares**.

INFORMATIONAL-05	Permissionless VaultV2.setDelegator() and factory create()	Acknowledged

Description

Lines:

- VaultV2.sol#L427
- VaultConfigurator.sol#L31

VaultV2.setDelegator() has no access control. **DelegatorFactory.create()** is also permissionless, so anyone can deploy a delegator pre-bound to a victim vault with attacker-controlled roles. **VaultConfigurator.create()** deploys the vault, deploys the delegator, and calls **VaultV2.setDelegator()**, but it has no access control either - any address can invoke it.

Recommendation

We recommend adding access control to **DelegatorFactory.create()** and **VaultConfigurator.create()** via a dedicated factory role, so only authorized deployers can mint and wire vault/delegator pairs. Optionally restrict **VaultV2.setDelegator()** to the vault admin (**DEFAULT_ADMIN_ROLE**).

Client's comments

Intended to be deployed through VaultConfigurator.

INFORMATIONAL-06	WithdrawalQueue.requestRedeem uses unsafe _mint and never burns NFT after full claim	Acknowledged

Description

Lines: WithdrawalQueue.sol#L103-L130

WithdrawalQueue.requestRedeem() (line 114) uses **_mint(receiver, tokenId)** rather than **_safeMint**. A contract receiver that does not implement **ERC721Receiver** loses access to the redemption NFT — but the user's vault shares were already pulled in via **safeTransferFrom**. Funds are effectively trapped.

Separately, **WithdrawalQueue.claim()** never burns the NFT after full claim (**sharesClaimed == shares**). The transferable position persists; subsequent transfers + **WithdrawalQueue.claim()** calls emit zero-value **Claim** events to the new owner, leading to off-chain confusion and storage bloat.

Recommendation

We recommend using **_safeMint** so contract recipients without **onERC721Received** revert (no shares lost) and burning the NFT in **WithdrawalQueue.claim()** or disallowing zero-value claims.

Client's comments

Seems to be making integrations more complex than safe.

INFORMATIONAL-07	UniversalDelegator.swapAdapters lacks adapter validation and a self-swap guard	Acknowledged

Description

Lines: UniversalDelegator.sol#L224-L237

UniversalDelegator.swapAdapters() does not validate its arguments with **_isAdapterAdded()**. When an address is not in **adapters**, its position stays **type(uint256).max** and the indexed assignment reverts on an out-of-bounds access, so validation relies on an implicit array revert rather than an explicit check. The function also does not reject **adapter1 == adapter2**, so a self-swap performs a no-op while still emitting **SwapAdapters**.

Recommendation

We recommend validating both adapters and rejecting a self-swap.

Client's comments

> Prefer minimalistic code style.

INFORMATIONAL-08	AppAdapter.slash() does not call VaultV2.accrueInterest()	Acknowledged

Description

Lines: AppAdapter.sol#L103-L124

AppAdapter.slash() updates the adapter stake, calls **UniversalDelegator.decreaseLimits()**, and sends assets to the burner, but does not call **VaultV2.accrueInterest()** first. Fees for the elapsed pre-slash window are therefore charged on the post-slash TVL when accrual eventually runs elsewhere.

Recommendation

We recommend calling **VaultV2.accrueInterest()** at the start of **AppAdapter.slash()** for the pre-slash accounting baseline.

Client's comments

> Leaving AppAdapter as a usual adapter here, so not adding explicit accrueInterest.

INFORMATIONAL-09	UniversalDelegator.removeAdapter() can be blocked by dust	Acknowledged

Description

Lines: UniversalDelegator.sol#L174-L176

UniversalDelegator.removeAdapter() requires **IAdapter.totalAssets() == 0**. A 1-wei underlying donation to the adapter reverts with **AdapterHasAssets()** until the dust is swept.

Recommendation

We recommend allowing removal when dust remains, or calling **UniversalDelegator.sweepPending()** before removal.

Client's comments

> To be covered externally, e.g., from frontend via multicall.

INFORMATIONAL-10	1 wei WithdrawalQueue.requestRedeem() can block VaultV2.withdraw() / VaultV2.redeem()	Acknowledged

Description

Lines:

- VaultV2.sol#L318-L320
- WithdrawalQueue.sol#L103-L119
- WithdrawalQueue.sol#L133-L152

VaultV2._withdraw() reverts with **PendingWithdrawalQueue()** while **UniversalDelegator.sweepPending() > 0**. Anyone can call **WithdrawalQueue.requestRedeem()** with 1 wei of shares; while the queue cannot be fully filled, all direct **withdraw()**/ **redeem()** calls revert for every user.

Recommendation

We recommend enforcing a minimum **WithdrawalQueue.requestRedeem()** size.

Client's comments

Intended behaviour, as non-instant withdrawals can be any number of shares.

INFORMATIONAL-11	Withdrawal surfaces insufficient liquidity as generic ERC20 balance error	Acknowledged

Description

Line: VaultV2.sol#L322-L326

VaultV2._withdraw() calls **UniversalDelegator.onWithdraw()**, which runs a best-effort **UniversalDelegator._deallocateAll()** loop and does not revert when adapters return less than requested (e.g. **AppAdapter._deallocate()** always returns 0). **VaultV2._withdraw()** then calls **super._withdraw()** without checking that **freeAssets() >= assets**, so when vault liquidity is still short the ERC4626 payout reverts with OpenZeppelin's **ERC20InsufficientBalance** instead of a vault-specific error. **InsufficientFreeAssets()** is already declared in IVaultV2 but is never used.

Recommendation

We recommend asserting vault liquidity after **UniversalDelegator.onWithdraw()** and reverting with the existing custom error before calling **super._withdraw()**:

```
    if (toWithdraw > 0) {
        UniversalDelegator(delegator).onWithdraw(toWithdraw);
    }
+   if (freeAssets() < assets) {
+       revert InsufficientFreeAssets();
+   }
    super._withdraw(caller, receiver, owner, assets, shares);
```

Client's comments

Prefer a more minimalistic and optimised approach.

INFORMATIONAL-12	Post-withdrawal free assets are not re-allocated to adapters	Acknowledged

Description

Lines:

- VaultV2.sol#L326-L327
- VaultV2.sol#L289-L296
- UniversalDelegator.sol#L385-L387

VaultV2._deposit() ends with **UniversalDelegator.onDeposit()**, which calls **UniversalDelegator._allocateAll(type(uint256).max)** and deploys vault free assets into adapters. **VaultV2._withdraw()** has no symmetric step: it may call **UniversalDelegator.sweepPending()**, which deallocates each adapter's full **Adapter.freeAssets()**, and/or **UniversalDelegator.onWithdraw()** via **UniversalDelegator._deallocateAll()**, then pays out only **assets** and updates **_totalAssets** without re-allocating the remainder. On **AppAdapter**, **deallocate()** can return the entire non-slashable balance when the requested amount is smaller, so excess funds land in the vault idle until a later deposit or manual allocation.

Impact: Leftover vault balance sits undeployed after withdrawals, reducing yield and diverging from the deposit path's "allocate all free assets" behavior. For AppAdapter-backed vaults, non-slashable funds pulled out during withdrawal settlement may remain in the vault without slash protection until they are explicitly re-allocated.

Recommendation

We recommend mirroring the deposit path by calling **UniversalDelegator._allocateAll()** after a successful withdrawal when no withdrawal-queue backlog remains.

Client's comments

> Considered to add some onWithdrawComplete at the end of _withdraw(), but decided that it's not that important to add a new function and increase gas costs.

INFORMATIONAL-13	Zero-amount allocation still invokes vault pull and interest accrual	Fixed at: 98c4a71

Description

Line: UniversalDelegator.sol#L478

UniversalDelegator._allocate() caps **assets** with **UniversalDelegator.allocatable(adapter)** but does not return early when the result is zero, and still calls **VaultV2.pull()**, which always runs **accrueInterest()** before transferring. This occurs whenever an adapter is at its limit, vault free assets are zero, or a curator passes **assets = 0** via **UniversalDelegator.allocate()** / **UniversalDelegator._allocateAll()**. **Adapter.allocate()** and **UniversalDelegator._deallocate()** both skip work on zero amounts; **UniversalDelegator._allocate()** does not.

Impact: No-op allocation paths pay extra gas and may run **VaultV2.accrueInterest()** and emit **Allocate** with **allocated = 0** once per capped adapter in an **UniversalDelegator._allocateAll()** loop, without moving funds.

Recommendation

We recommend adding an early return before **VaultV2.pull()**, consistent with **UniversalDelegator._deallocate()**:

```
    function _allocate(address adapter, uint256 assets) internal returns (uint256 allocated) {
        assets = Math.min(assets, allocatable(adapter));
+       if (assets == 0) {
+           return 0;
+       }
        VaultV2(vault).pull(assets, adapter);
        allocated = IAdapter(adapter).allocate(assets);
```

INFORMATIONAL-14	Delegator address is fetched on every use instead of being cached	Acknowledged

Description

Lines:

- AppAdapter.sol#L117
- AppAdapter.sol#L140
- AppAdapter.sol#L175
- Adapter.sol#L34
- WithdrawalQueue.sol#L118

IVaultV2(vault).delegator() is called via external call every time the delegator address is needed — in **AppAdapter.slash()**, **AppAdapter.release()**, **AppAdapter._requestDeallocate()**, the **Adapter.onlyDelegator** modifier, and **WithdrawalQueue.requestRedeem()**. Each call incurs an SLOAD on the adapter contract to read **vault**, followed by a cross-contract call to read **delegator** from vault storage. **VaultV2.setDelegator()** enforces that the delegator address is set exactly once and never changes, so all these live lookups are redundant.

Recommendation

We recommend caching the **delegator** address in the **Adapter** base contract during **__initialize**, storing it alongside **vault**:

```
+ address public delegator;

  function __initialize(...) internal override {
      vault = initVault;
+     delegator = IVaultV2(initVault).delegator();
  }
```

Client's comments

Less caching allows more flexibility for future updates.

INFORMATIONAL-15	VaultV2.redeemable() rounds shares up while redeem floors assets, so redeem(redeemable()) can revert at the boundary	Fixed at: 446b636

Description

Lines:

- VaultV2.sol#L221-L224
- VaultV2.sol#L311-L328

VaultV2.redeemable() returns **previewWithdraw(withdrawable())**. Let's assume (**offset = _decimalsOffset()**):

- **W = withdrawable()** — assets the vault can realise instantly;
- **n = totalSupply() + 10**offset**, **d = totalAssets() + 1** — the OZ ERC4626 virtual conversion terms (**d/n** is the assets-per-share price);
- **s** — the resulting share count; **a** — the assets **redeem** releases for those shares.

previewWithdraw rounds shares up (OZ **Ceil**): **s = ceil(W·n/d)**. The caller's **VaultV2.redeem(s)** releases assets via **previewRedeem**, which rounds down (**Floor**): **a = floor(s·d/n)**. The ceil-then-floor round-trip can yield **a > W**; let **delta = a – W** be the overshoot.

With the remainder **r = (W·n) mod d**: **r = 0** gives **a = W**; **r > 0** gives **delta = floor((d – r)/n)**. Hence **delta ≥ 1 ⟺ 0 < r ≤ d – n**, which requires **d > n** (price above one — an accrued-yield vault).

The overshoot reverts at the capacity boundary. **redeem(s)** enters **VaultV2._withdraw()** with **assets = W + delta**, but **withdrawable() = freeAssets() + deallocatable()** is already the most the vault can produce in one call, so **UniversalDelegator.onWithdraw()** cannot source the extra **delta** and **super._withdraw** reverts transferring **W + delta** against **W**. In atomic same-transaction use the revert is unconditional whenever **delta ≥ 1**; it is avoided only if **freeAssets()** grew (e.g. a new deposit) between the **redeemable()** read and the **redeem()** call. (**redeem** runs **accrueInterest()** first, so **delta** is evaluated at redeem-time state.)

Example: **totalSupply()=9, totalAssets()=12** (offset 0 → **n=10, d=13**, price 1.3), **withdrawable()=4** → **redeemable() = ceil(40/13) = 4**; **redeem(4)** releases **floor(52/10) = 5 > 4** → revert (**delta = 1**).

Impact: **redeemable()** is supposed to be the upper bound for instant redemption, but **redeem(redeemable())** can revert at the boundary.

Recommendation

We recommend computing **redeemable()** with floor rounding so the round-trip never exceeds **withdrawable()**:

```
 function redeemable() public returns (uint256) {
-    return previewWithdraw(withdrawable());
+    return convertToShares(withdrawable());
 }
```

INFORMATIONAL-16	VaultV2.withdrawable()/redeemable() overstate instant capacity by ignoring the withdrawal queue's prior claim	Acknowledged

Description

Lines:

- VaultV2.sol#L216-L219
- VaultV2.sol#L221-L224
- WithdrawalQueue.sol#L133-L152

VaultV2.withdrawable() returns **freeAssets() + UniversalDelegator.deallocatable()** and **VaultV2.redeemable()** wraps it. Neither subtracts **WithdrawalQueue.pendingAssets()**, but **VaultV2._withdraw()** serves the queue first (via **sweepPending**), so a new instant withdrawer can realise only **withdrawable() – pendingAssets()**. An instant **withdraw**/**redeem** sized against the quoted figure can revert while the queue is non-empty.

Subtracting **pendingAssets()** inside **withdrawable()** itself would break the queue: **WithdrawalQueue.fill()** caps its own fill, so removing the queue's pending from that cap is circular — at liquidity == pending it yields 0 and the queue never fills.

Impact: When the queue holds unfilled requests, both views return a number larger than any single **withdraw**/**redeem** can realise.

Recommendation

We recommend:

1. Split the raw liquidity (**fill()**'s capacity) from the user-facing quote (pending-adjusted):

```
function realisableLiquidity() public returns (uint256) {          // raw — today's withdrawable()
    return freeAssets() + UniversalDelegator(delegator).deallocatable();
}
function withdrawable() public returns (uint256) {                 // instant capacity after the queue
    return realisableLiquidity().saturatingSub(WithdrawalQueue(withdrawalQueue).pendingAssets());
}
```

2. Point **fill()** at the raw primitive so the queue still fills itself:

```
-       shares = Math.min(shares, IERC4626(vault).previewDeposit(VaultV2(vault).withdrawable()));
+       shares = Math.min(shares, IERC4626(vault).previewDeposit(VaultV2(vault).realisableLiquidity()));
```

Client's comments

To be covered externally, e.g., from the frontend.

INFORMATIONAL-17	Released assets remain in the adapter until an external trigger moves them to the vault	Acknowledged

Description

Line: AppAdapter.sol#L127

AppAdapter.release() accounts for the released amount by incrementing **slashed** on the current stake (reducing **AppAdapter._slashable()**) and calls **UniversalDelegator.decreaseLimits()** on the delegator to prevent new allocations above the adjusted slashable balance. However, the physical tokens remain in the adapter — no transfer to the vault is performed. Since **AppAdapter.freeAssets()** returns **totalAssets() – slashable()**, the released amount immediately becomes **freeAssets()** . Moving those assets to the vault requires an independent trigger: **UniversalDelegator.sweepPending()** call or **UniversalDelegator.deallocate()** call. By contrast, **AppAdapter.slash()** explicitly transfers the slashed tokens to the burner in the same call.

Recommendation

We recommend calling **sweepPending()** at the end of **release()** to immediately move the newly freed assets to the vault and fill any pending withdrawal queue:

```
IUniversalDelegator(delegator)
    .decreaseLimits(IUniversalDelegator(delegator).absoluteLimitOf(address(this)).saturatingSub(_slashable()), 0);
+ IUniversalDelegator(delegator).sweepPending();

emit Release(amount);
```

Client's comments

Funds are still swept during any further allocate/withdraw actions.

INFORMATIONAL-18	UniversalDelegator.deallocate() does not verify adapter is registered	Fixed at: c7aaee4

Description

Lines: UniversalDelegator.sol#L284-L292

UniversalDelegator.deallocate() calls **UniversalDelegator._deallocate()** without checking **_isAdapterAdded[adapter]**. A holder of **DEALLOCATE_ROLE** can pass any address that implements **IAdapter**, even if that adapter is not in the delegator route.

Recommendation

We recommend adding **if (!_isAdapterAdded[adapter]) revert IUniversalDelegator.InvalidAdapter();** at the start of **UniversalDelegator.deallocate()**.

INFORMATIONAL-19	Missing reentrancy protection on the deallocation path exposes share pricing to reentrancy from integrated protocols	Fixed at: 4d42fa7

Description

Line: UniversalDelegator.sol#L381

UniversalDelegator.sweepPending() is permissionless and not marked **nonReentrant**, yet it deallocates from adapters via **UniversalDelegator._deallocate()**, which calls the external protocol behind each adapter (e.g. **IAaveV3Pool.withdraw()** / **IMorphoVaultV2.withdraw()**). Neither **VaultV2** nor **WithdrawalQueue** apply a reentrancy guard, and share price is derived from a live **totalAssets()** read in **VaultV2.getAccrueInterest()** that sums adapter-reported balances. If an adapter's external protocol transfers control to the caller after reducing the adapter's reported position but before delivering the underlying, the live **VaultV2.totalAssets()** is transiently understated; because this path holds no delegator lock, a reentrant **VaultV2.deposit()** acquires the **UniversalDelegator.onDeposit()** lock fresh and mints shares against the deflated value, and queue settlement via **WithdrawalQueue.fill()** may occur at the same deflated price.

The currently integrated protocols do not meet this precondition: Aave V3 burns and transfers within **AToken.burn()** with no intervening callback, and Morpho Vault V2 yields control (gates, liquidity adapter) only before burning the adapter's shares, so the adapter's reported value remains intact during those callbacks. The protection is nonetheless absent, so the behavior may become reachable for any present or future adapter whose protocol yields control to the withdrawer mid-withdrawal, or for a non-standard (hook-bearing) asset.

Impact: Under the conditions above, a reentrant deposit mints shares at an understated **VaultV2.totalAssets()**, diluting existing depositors and allowing value extraction, and an honest withdrawal request may be settled for less than its fair value. Exploitability is conditional on the call ordering of the integrated protocol behind an adapter.

Recommendation

We recommend applying **nonReentrant** to **UniversalDelegator.sweepPending()** and to the **VaultV2** and **WithdrawalQueue** entry points (**deposit()**/**mint()**/**withdraw()**/**redeem()** and **requestRedeem()**/**claim()**/**fill()**), ideally under a shared cross-contract reentrancy lock, so share-price integrity does not depend on the internal call ordering of integrated protocols.

INFORMATIONAL-20	Adapter.deallocate can return more than the requested amount when free assets exceed it	Acknowledged

Description

Line: Adapter.sol#L69-L72

Adapter.deallocate() is meant to free up to **amount** assets and return how much it actually freed. When **curFreeAssets >= amount**, the current code over-returns: the ternary takes the **: 0** arm and the function returns the entire **curFreeAssets**, not the requested **amount**. **freeAssets()** is the adapter's idle token balance (**Adapter.freeAssets()**), so any amount smaller than that balance yields a return value strictly greater than what was asked.

The delegator trusts this return value. **UniversalDelegator._deallocate()** pushes the full returned amount back into the vault. **VaultV2.push()** executes **safeTransferFrom(adapter, vault, deallocated)**. Because the over-returned figure equals the adapter's real idle balance, the transfer is funded and does not revert; the adapter simply pushes more liquid assets into the vault than the caller requested.

Impact: **forceDeallocate** clamps the absolute limit harder than intended. In **UniversalDelegator.forceDeallocate()**, with **adapterTotalAssets = 100** and a request of **50** against an adapter holding **80** free, **deallocated = 80**. The **if (deallocated < assets)** branch is skipped (**80 < 50** is false), so **pending = 0**, and the new absolute limit becomes **min(absoluteLimitOf, 100 − 80 − 0) = 20**. A curator intending to free 50 of liquid capacity ends up with the absolute limit forced down to 20 instead of the intended 50.

Recommendation

We recommend capping the free-assets contribution at the requested amount so the function never returns more than **amount**, leaving any surplus idle balance in the adapter for the next call:

```
 function deallocate(uint256 amount) public onlyDelegator returns (uint256) {
     uint256 curFreeAssets = freeAssets();
-    return curFreeAssets + (curFreeAssets < amount ? _deallocate(amount - curFreeAssets) : 0);
+    if (curFreeAssets >= amount) {
+        return amount;
+    }
+    return curFreeAssets + _deallocate(amount - curFreeAssets);
 }
```

Client's comments

Intended.

INFORMATIONAL-21	VaultV2.balanceOf and share transfers do not reflect pending fee shares	Acknowledged

Description

Lines: VaultV2.sol#L149-L150

Fee shares are minted to their receivers inside **VaultV2.accrueInterest()**, which calls **_mint()** for **managementFeeReceiver**, **performanceFeeReceiver**, and **lastProtocolFeeReceiver**. Between two accruals, the unminted fee shares exist conceptually but are not yet written to the receivers' balance checkpoints. **VaultV2.balanceOf()** returns the stored checkpoint value directly without accounting for this pending amount.

The same staleness affects share transfers. **VaultV2._update()** reads **_balances[from].latest()** and **_balances[to].latest()** and does not call **accrueInterest()**.

Recommendation

We recommend documenting this behaviour, clarifying that **balanceOf()** and share transfers reflect only minted shares and that fee-receiver positions are only up to date after **accrueInterest()** has run. Fee receivers and integrators acting on their behalf should batch such calls (transfers, redemptions, balance reads) in the same transaction as **VaultV2.accrueInterest()** so that pending fee shares are materialised beforehand.

Client's comments

Current approach simplifies the code and does not introduce rebase-tokens-like behaviour for integrations.

INFORMATIONAL-22	Instant withdraw deallocates over every adapter twice — once for the queue, once for the caller	Acknowledged

Description

Lines:

- VaultV2.sol#L311-L328
- UniversalDelegator.sol#L381-L426
- UniversalDelegator.sol#L370-L376
- UniversalDelegator.sol#L469-L475

On an instant withdraw, **VaultV2._withdraw()** calls **UniversalDelegator.sweepPending()** — which runs **_deallocateAll(queueGap)** over **adapters[]** to fill the queue — and then **UniversalDelegator.onWithdraw()**, which runs **UniversalDelegator._deallocateAll()** again for the caller's amount. The total to free (**queueGap + toWithdraw**) is known up front, so one pass would suffice; the second full traversal of **adapters[]** is redundant.

Impact: Per-call gas on withdraw/redeem (and deposit/**requestRedeem**, which also route through **sweepPending**) scales with adapter count and per-adapter view cost

Recommendation

We recommend two changes:

1. **UniversalDelegator** — one parametrized sweep. Factor the body of **sweepPending()** into **_sweep(uint256 extraAssets)** and change only the deallocation target to cover the queue gap and the caller's amount together:

```
-function sweepPending() public returns (uint256 pendingAssets) {
-    // ... idle-balance harvest loop ...
-    _deallocateAll(WithdrawalQueue(wq).pendingAssets().saturatingSub(VaultV2(vault).freeAssets()));
-    WithdrawalQueue(wq).fill();
-    // ... adaptersWithPending rebuild + reset loop ...
-}
+function sweepPending() public returns (uint256 pendingAssets) {
+    return _sweep(0);
+}
+function sweepPendingFor(uint256 extraAssets) external returns (uint256 pendingAssets) {
+    if (msg.sender != vault) revert NotVault();
+    return _sweep(extraAssets);
+}
+function _sweep(uint256 extraAssets) internal returns (uint256 pendingAssets) {
+    // ... idle-balance harvest loop (unchanged) ...
+    _deallocateAll(
+        (WithdrawalQueue(wq).pendingAssets() + extraAssets).saturatingSub(VaultV2(vault).freeAssets())
+    );
+    WithdrawalQueue(wq).fill();
+    pendingAssets = WithdrawalQueue(wq).pendingAssets();
+    // ... adaptersWithPending rebuild + reset loop (unchanged) ...
+}
```

With **extraAssets = 0** the deallocation target is **pendingAssets().saturatingSub(freeAssets())** — byte-for-byte today's **sweepPending()** — so every other caller (**onDeposit**, **deallocate***, **forceDeallocate**, permissionless) is unaffected.

2. **VaultV2._withdraw()** — route the instant path through the combined sweep and drop the second pass. Pass the caller's full **assets** (not **assets.saturatingSub(freeAssets())**): the queue has priority and consumes the vault's existing free assets first, so the caller's portion must be freed in full.

```
 function _withdraw(address caller, address receiver, address owner, uint256 assets, uint256 shares) internal override {
-    if (withdrawalQueue != msg.sender) {
-        if (UniversalDelegator(delegator).sweepPending() > 0) {
-            revert PendingWithdrawalQueue();
-        }
-    }
-    uint256 toWithdraw = assets.saturatingSub(freeAssets());
-    if (toWithdraw > 0) {
-        UniversalDelegator(delegator).onWithdraw(toWithdraw);
-    }
+    if (withdrawalQueue != msg.sender) {
+        // one combined deallocation pass: queue gap + caller's full amount
+        if (UniversalDelegator(delegator).sweepPendingFor(assets) > 0) {
+            revert PendingWithdrawalQueue();
+        }
+    } else {
+        // queue's own redeem recursion — unchanged
+        uint256 toWithdraw = assets.saturatingSub(freeAssets());
+        if (toWithdraw > 0) {
+            UniversalDelegator(delegator).onWithdraw(toWithdraw);
+        }
+    }
     super._withdraw(caller, receiver, owner, assets, shares);
     _totalAssets -= assets;
 }
```

Client's comments

Prefer minimalistic code style.

INFORMATIONAL-23	Bulk allocation sweeps all adapters twice per auto-allocate adapter, making onDeposit/allocateAll O(autoAllocateAdapters × adapters)	Fixed at: 98c4a71

Description

Lines:

- UniversalDelegator.sol#L460-L466
- UniversalDelegator.sol#L478-L488
- UniversalDelegator.sol#L108-L120
- UniversalDelegator.sol#L96-L100
- VaultV2.sol#L266-L275

UniversalDelegator._allocateAll() loops **autoAllocateAdapters**, and each iteration's **UniversalDelegator._allocate()** performs two independent full O(N) sweeps over **adapters[]**:

1. Limit sweep — **UniversalDelegator.allocatable()** → **UniversalDelegator.limitOf()** → **VaultV2.totalAssets()** → **UniversalDelegator.totalAssets()**, which sums every adapter's **IAdapter.totalAssets()**.
2. Pull sweep — **_allocate** then calls **VaultV2.pull()** unconditionally (even after **allocatable()** clamps **assets** to 0), and **pull()** runs **VaultV2.accrueInterest()** → **getAccrueInterest()** → **UniversalDelegator.totalAssets()**, the same O(N) sweep again.

So the bulk path runs ~2 full adapter sweeps per auto-allocate adapter — O(A × N) external calls (A = **autoAllocateAdapters.length**, N = **adapters.length**; O(N²) when every adapter is auto-allocate), and an adapter whose **allocatable()** is 0 still pays the pull sweep (plus a **pull(0)** and the event's **totalAssets()** read) for nothing.

VaultV2.totalAssets() is invariant across the loop up to per-adapter rounding (a **pull** only moves assets vault→adapter, leaving **freeAssets() + delegator.totalAssets()** unchanged; AaveV3/Morpho **totalAssets()** round down by ≤1 wei, exact for AppAdapter), so both sweeps are redundant.

Impact: Gas on **onDeposit**/**allocate**/**allocateAll** scales quadratically with adapter count and per-adapter view cost.

Recommendation

Three independent reductions; together they take **onDeposit**/**allocateAll** from O(A × N) to O(N + A).

1. Cache **totalAssets()** (removes the limit sweep). Read **VaultV2.totalAssets()** once before the loop and thread it into the limit math; track the vault free balance locally (it changes per **pull**):

```
 function _allocateAll(uint256 assets) internal returns (uint256 allocated) {
+    uint256 cachedTotalAssets = VaultV2(vault).totalAssets(); // invariant up to per-adapter rounding
+    uint256 freeRemaining = VaultV2(vault).freeAssets();
     for (uint256 i; i < autoAllocateAdapters.length && assets > 0; ++i) {
-        uint256 curAllocated = _allocate(autoAllocateAdapters[i], assets);
+        uint256 curAllocated = _allocate(autoAllocateAdapters[i], assets, cachedTotalAssets, freeRemaining);
         allocated += curAllocated;
         assets -= curAllocated;
+        freeRemaining -= curAllocated;
     }
 }
```

Add overloads of **_allocate**, **allocatable**, and **limitOf** that take the cached **(totalAssets_, freeRemaining)** instead of re-reading **VaultV2.totalAssets()**/**freeAssets()**; keep the public views unchanged for external callers.

2. Zero-guard **_allocate** (removes the pull sweep for empty adapters). Return early when the clamped **assets** is 0, so adapters that can take nothing skip **pull**/**accrueInterest**/**push**/**emit**:

```
 function _allocate(address adapter, uint256 assets, uint256 totalAssets_, uint256 freeRemaining) internal returns
(uint256) {
    assets = Math.min(assets, allocatable(adapter, totalAssets_, freeRemaining));
+    if (assets == 0) return 0;
    VaultV2(vault).pull(assets, adapter);
    ...
 }
```

3. Remove the pull sweep for non-empty allocations by accruing once per operation. **VaultV2.pull()** calls **accrueInterest()** on every invocation, but a **pull** is value-neutral for fees (it only moves assets vault→adapter), so accruing per pull is redundant. Drop the **accrueInterest()** from **pull()** and accrue once at the allocation-operation entry — **VaultV2._deposit()** already does this ahead of **onDeposit**; add a single **accrueInterest()** at the start of **allocate**/**allocateAll**.

Client's comments

Partially fixed

INFORMATIONAL-24	RestakingAppAdapter.slash() and RestakingAppAdapter.release() unconditionally call syncSlash()	Acknowledged

Description

Lines:

- RestakingAppAdapter.sol#L165-L168
- RestakingAppAdapter.sol#L171-L174

Both **RestakingAppAdapter.slash()** and **RestakingAppAdapter.release()** always call **RestakingAppAdapter.syncSlash()** before proceeding. **RestakingAppAdapter.syncSlash()** only does useful work when there are pending slash redemptions to claim (**RestakingAppAdapter.isUnsyncedSlash() == true**, i.e. **firstUnclaimed < tokenIds.length**).

Recommendation

We recommend calling **RestakingAppAdapter.syncSlash()** only when needed:

```
if (isUnsyncedSlash()) {
    syncSlash();
}
```

in both **RestakingAppAdapter.slash()** and **RestakingAppAdapter.release()**.

Client's comments

Prefer code simplicity here.

INFORMATIONAL-25	RestakingAppAdapter does not cache underlyingVaults.length in loops	Fixed at: a233b69

Description

Lines:

- RestakingAppAdapter.sol#L77
- RestakingAppAdapter.sol#L98
- RestakingAppAdapter.sol#L117
- RestakingAppAdapter.sol#L131
- RestakingAppAdapter.sol#L180
- RestakingAppAdapter.sol#L204
- RestakingAppAdapter.sol#L212

Several functions iterate over **underlyingVaults** using **underlyingVaults.length** directly in the loop condition, causing repeated storage reads on each iteration. The array length is bounded by **MAX_DEPTH** and does not change during these loops, so it can be cached in a local variable.

Recommendation

We recommend caching the length before each loop, e.g.:

```
uint256 len = underlyingVaults.length;
for (uint256 i; i < len; ++i) { ... }
```

INFORMATIONAL-26	MerklClaimer constructor does not validate merklDistributor	Acknowledged

Description

Lines:

- MerklClaimer.sol#L17-L19

MerklClaimer stores **merklDistributor** as **immutable MERKL_DISTRIBUTOR** without checking for **address(0)**. The value cannot be corrected without redeploying the adapter.

Recommendation

We recommend adding a zero-address check in the constructor.

Client's comments

Prefer code simplicity here.

INFORMATIONAL-27	UniversalDelegator.allocateExact() deallocates from other adapters without checking target capacity	Acknowledged

Description

Lines:

- **UniversalDelegator.sol#L267-L281**

UniversalDelegator.allocateExact() ensures **freeAssets >= assets** via **UniversalDelegator._deallocateAll()**, but **UniversalDelegator._allocate()** may allocate far less due to **limitOf(adapter) - totalAssets** and **IAdapter(adapter).allocatable()**. Trusted **ALLOCATE_ROLE** callers can unintentionally deallocate from other adapters without allocating to the target. Capital sits in **freeAssets** instead of the intended adapter; the "exact" semantics are not met.

Recommendation

We recommend computing the target's max allocatable amount before deallocation and only calling **UniversalDelegator._deallocateAll()** for the amount actually needed.

Client's comments

Acknowledged.

INFORMATIONAL-28	Token-ban checks in convert() and syncReward() perform external asset() calls that the immutable vault chain already determines	Fixed at: 4b9f088

Description

Lines:

- RestakingAppAdapter.sol#L95
- RestakingAppAdapter.sol#L98-L102
- RestakingAppAdapter.sol#L120

RestakingAppAdapter.convert() bans selling any token of the restaking chain by calling **IERC4626(...).asset()** on the vault and on every **underlyingVaults[i]**, and **RestakingAppAdapter.syncReward()** reads **IERC4626(vault).asset()** once per iteration.

Define **W0 = underlyingVaults[0] = IERC4626(vault).asset()** (the wrapper the adapter holds), **Wk = underlyingVaults[k]**, and **B = asset** (the configured base asset). **RestakingAppAdapter.__initialize()** builds the chain so that **IERC4626(underlyingVaults[i]).asset() == underlyingVaults[i+1]** and **IERC4626(underlyingVaults[last]).asset() == asset**, and both **underlyingVaults** and **asset** are immutable after initialization. The banned set therefore equals **{W0, ..., W(n-1), B}**, which is exactly **underlyingVaults** together with **asset**. Each **asset()** call in **convert()** and **syncReward()** consequently returns a value the contract already stores.

Impact: Up to **n + 1** external **STATICCALL**s per **RestakingAppAdapter.convert()** invocation and one per **RestakingAppAdapter.syncReward()** iteration are spent retrieving values that are derivable from storage.

Recommendation

We recommend comparing **tokenIn** against the stored **underlyingVaults[i]** (which are **W0..W(n-1)**) and **asset** (which is **B**) directly in **RestakingAppAdapter.convert()**:

```
-       if (tokenIn == IERC4626(vault).asset()) {
+       if (tokenIn == asset) {
            revert InvalidTokenIn();
        }
        for (uint256 i; i < underlyingVaults.length; ++i) {
-           if (tokenIn == IERC4626(underlyingVaults[i]).asset()) {
+           if (tokenIn == underlyingVaults[i]) {
                revert InvalidTokenIn();
            }
        }
```

INFORMATIONAL-29	Converter registration does not reject the zero address	Acknowledged

Description

Lines:

- CoWSwapConverter.sol#L234-L237
- CoWSwapConverter.sol#L159-L163

CoWSwapConverter.__CoWSwapConverter_init() and **CoWSwapConverter.setConverters()** assign the supplied address array to storage without validating its entries. A zero address (or a duplicate) in the list is accepted silently. A zero entry is inert for access control — no caller is **address(0)** — but it reflects a configuration error that goes undetected and occupies a slot in the linear scan that **CoWSwapConverter._isConverter()** performs on every **convert()** call.
Impact: A malformed converter set is accepted without signal.

Recommendation

We recommend validating each entry in **CoWSwapConverter.__CoWSwapConverter_init()** and **CoWSwapConverter.setConverters()**, reverting when an entry is **address(0)** (and optionally de-duplicating the list).

Client's comments

Prefer code simplicity here.

INFORMATIONAL-30	prepareConvert() does not mirror the order validation performed in convert()	Fixed at: 6c2534c

Description

Lines:

- CoWSwapConverter.sol#L136-L154
- CoWSwapConverter.sol#L76-L106

CoWSwapConverter.prepareConvert() reserves a request hash and sets **executableAt[nonces(tokenIn)][requestHash] = block.timestamp + EXECUTION_DELAY** after checking only that the contract holds **amountIn** of **tokenIn**. It does not apply the checks that **CoWSwapConverter.convert()** enforces at execution: **tokenIn != tokenOut**, **params.buyAmount != 0**, and **block.timestamp < params.validTo <= block.timestamp + MAX_VALID_TO_DURATION**.
Impact: A prepared request can mature into a call that always reverts, so the caller waits the full **EXECUTION_DELAY** before discovering the reservation is unusable.

Recommendation

We recommend factoring the shared predicates into an internal validation helper invoked by both **CoWSwapConverter.prepareConvert()** and **CoWSwapConverter.convert()** to apply their **tokenIn**/**tokenOut** bans on the prepare path as well, so invalid parameters revert at preparation time rather than after the delay.

INFORMATIONAL-31	syncSlash() writes firstUnclaimed unconditionally even when the claim cursor did not advance	Acknowledged

Description

Line: RestakingAppAdapter.sol#L154

RestakingAppAdapter.syncSlash() reads **indexToClaim = requests.firstUnclaimed** and, after the inner claim loop, writes **requests.firstUnclaimed = indexToClaim** for every underlying vault. When no request is fully claimed in that pass — the loop breaks on the first not-yet-matured request, or runs zero iterations because **firstUnclaimed == tokenIds.length** — **indexToClaim** equals the value originally read, so the assignment stores the same value. **RestakingAppAdapter.syncSlash()** is permissionless and is frequently invoked when there is nothing to claim, so this same-value (warm) **SSTORE** executes once per underlying vault on the common idle path.

Impact: A redundant warm **SSTORE** (100 gas) per underlying vault per call on paths where the cursor does not move.

Recommendation

We recommend writing the cursor only when it advanced:

```
-         uint256 indexToClaim = requests.firstUnclaimed;
+         uint256 start = requests.firstUnclaimed;
+         uint256 indexToClaim = start;
          for (; indexToClaim < requests.tokenIds.length; ++indexToClaim) {
            // ...
          }
-         requests.firstUnclaimed = indexToClaim;
+         if (indexToClaim != start) {
+             requests.firstUnclaimed = indexToClaim;
+         }
```

Client's comments

Prefer code simplicity here.

INFORMATIONAL-32	Redundant base asset approval is set twice during RestakingAppAdapter initialization	Fixed at: 9c6ad8f

Description

Line: RestakingAppAdapter.sol#L245-L247

RestakingAppAdapter.__initialize() ends with an unconditional **forceApprove** of **curAsset** (the root base asset) to **underlyingVaults[underlyingVaults.length - 1]**, but the initialization loop already performs the same approval on its final iteration via **IERC20(IERC4626(curAsset).asset()).forceApprove(curAsset, type(uint256).max)** when **curAsset** is the deepest underlying vault. After a successful walk, **curAsset == params.asset**, so the post-loop block repeats an approval that is already in place and cannot change behavior.

Recommendation

We recommend removing the post-loop block at lines 273–276 and relying solely on the per-hop approval inside the loop, which already covers the base-asset → deepest-vault allowance needed by **RestakingAppAdapter.syncReward()**.

```
        if (curAsset != params.asset) {
            revert InvalidAsset();
        }
-       if (underlyingVaults.length > 0) {
-           IERC20(curAsset).forceApprove(underlyingVaults[underlyingVaults.length - 1], type(uint256).max);
-       }
    }
```

INFORMATIONAL-33	CoWSwapConverter.prepareConvert() does not validate validTo against the execution delay	Fixed at: 2c5891e

Description

Line: CoWSwapConverter.sol#L136-L154

CoWSwapConverter.prepareConvert() records **executableAt = block.timestamp + EXECUTION_DELAY** (**EXECUTION_DELAY** is 1 day) but never decodes or validates **OrderParams.validTo** from **data**. **CoWSwapConverter.convert()** only enforces **validTo > block.timestamp** and **validTo <= block.timestamp + MAX_VALID_TO_DURATION** (**MAX_VALID_TO_DURATION** is 30 minutes) at execution time. A **validTo** that is valid when **prepareConvert()** runs can already be expired once the delay elapses, so the permissionless **convert()** path reverts with **ExpiredOrder()**.

Recommendation

We recommend validating **validTo** inside **CoWSwapConverter.prepareConvert()** so the order remains valid at the earliest allowed execution time:

```
OrderParams memory params = abi.decode(data, (OrderParams));
uint48 earliest = uint48(block.timestamp) + EXECUTION_DELAY;
if (params.validTo <= earliest) revert ExpiredOrder();
```

INFORMATIONAL-34	Compromised converter can bypass the execution delay and settle orders at an attacker-chosen price	Acknowledged

Description

Lines: CoWSwapConverter.sol#L96-L102

In **CoWSwapConverter.convert()**, the **EXECUTION_DELAY** enforced through **CoWSwapConverter.prepareConvert()** is skipped when the caller is a registered converter. A converter can therefore pre-sign an order and make it executable within a single transaction, leaving no delay window in which **CoWSwapConverter.invalidateConvert()** could cancel it. Because **params.buyAmount** (the order limit price) is supplied by the caller and CoW Protocol settlement enforces only that the limit price is respected, a converter may submit an order with a near-zero **buyAmount**, causing the full **amountIn** to be sold for negligible output. The execution delay that mitigates this for permissionless callers does not apply here, so a single compromised converter key removes the only on-chain safeguard against an attacker-chosen price.

Impact: A compromised or malicious converter can sell the adapter's rewards balance at an arbitrarily low price to a colluding solver, with no delay window for cancellation, leading to loss of the converted funds.

Recommendation

We recommend requiring that every address registered via **CoWSwapConverter.setConverters()** (and the initial set in **__CoWSwapConverter_init()**) be a multisig or similarly hardened account, so that no single compromised key can pre-sign and immediately execute an order.

Client's comments

Acknowledged.

INFORMATIONAL-35	Missing documentation for the withdrawalRequests mapping and WithdrawalRequests struct	Fixed at: 505651c

Description

Lines:

- RestakingAppAdapter.sol#L32-L38

The **withdrawalRequests** mapping and its **WithdrawalRequests** struct carry no NatSpec and are not declared in **IRestakingAppAdapter**, unlike the adjacent **underlyingVaults** and the other documented interface members, so the non-obvious **firstUnclaimed** cursor and **tokenIds** backlog semantics are left undocumented.

Recommendation

We recommend documenting both the mapping and the struct.

INFORMATIONAL-36	Unguarded requestRedeem() lets a reverting underlying queue brick slash() and release()	Acknowledged

Description

Lines:

- RestakingAppAdapter.sol#L136-L140
- RestakingAppAdapter.sol#L183

RestakingAppAdapter moves slash proceeds down the restaking chain by alternating **requestRedeem()** (open a withdrawal at the current level) with **claim()** (collect the proceeds). The **claim()** calls are defensively wrapped in **try/catch**, but the adjacent **requestRedeem()** calls are not, so a single reverting underlying withdrawal queue propagates up and reverts the whole transaction.

In **syncSlash()** the redeem leg is unguarded while the claim leg below it is wrapped:

```
// Create a request for previously claimed vault shares.
if (amount > 0) {
    withdrawalRequests[vault].tokenIds
        .push(uint64(IWithdrawalQueue(withdrawalQueue).requestRedeem(amount, address(this)))); // not wrapped
    amount = 0;
}
// Try claim existing requests greedily.
...
try IWithdrawalQueue(withdrawalQueue).claim(tokenId, address(this)) returns (uint256 assets, uint256) {
    amount += assets;
} catch {}
```

The **_sendToBurner** override has the same asymmetry: **requestRedeem()** is called directly, while the following **claim()** is wrapped in **try/catch**.

slash() and **release()** both begin with **syncSlash()**, and base **slash()** also reaches the overridden **_sendToBurner()**. **requestRedeem()** is more than a queue call: it pulls vault shares and then runs **UniversalDelegator.sweepPending()**, so a revert anywhere in that chain bricks the whole **slash()** / **release()**.

Impact: The penalty mechanism can be made unavailable based on the state of numerous external contracts.

Recommendation

We recommend wrapping **requestRedeem()** in try/catch consistently with claim, defer failed redemptions, and ensure slash accounting can proceed even if a redemption fails.

Client's comments

Added docs.

STATE
MIND